

Chapter 13

Manipulating Feature Data

This chapter will show how to read and write attribute data, modify attribute tables, as well as create new features in feature classes. It will use three examples, one to create a custom identify tool, one to calculate polygon feature area, and one to create a bus route feature class from a GPS data file, to demonstrate these capabilities. Several related techniques including spatial filter, various types of cursors, and feature buffer will be discussed and practiced.

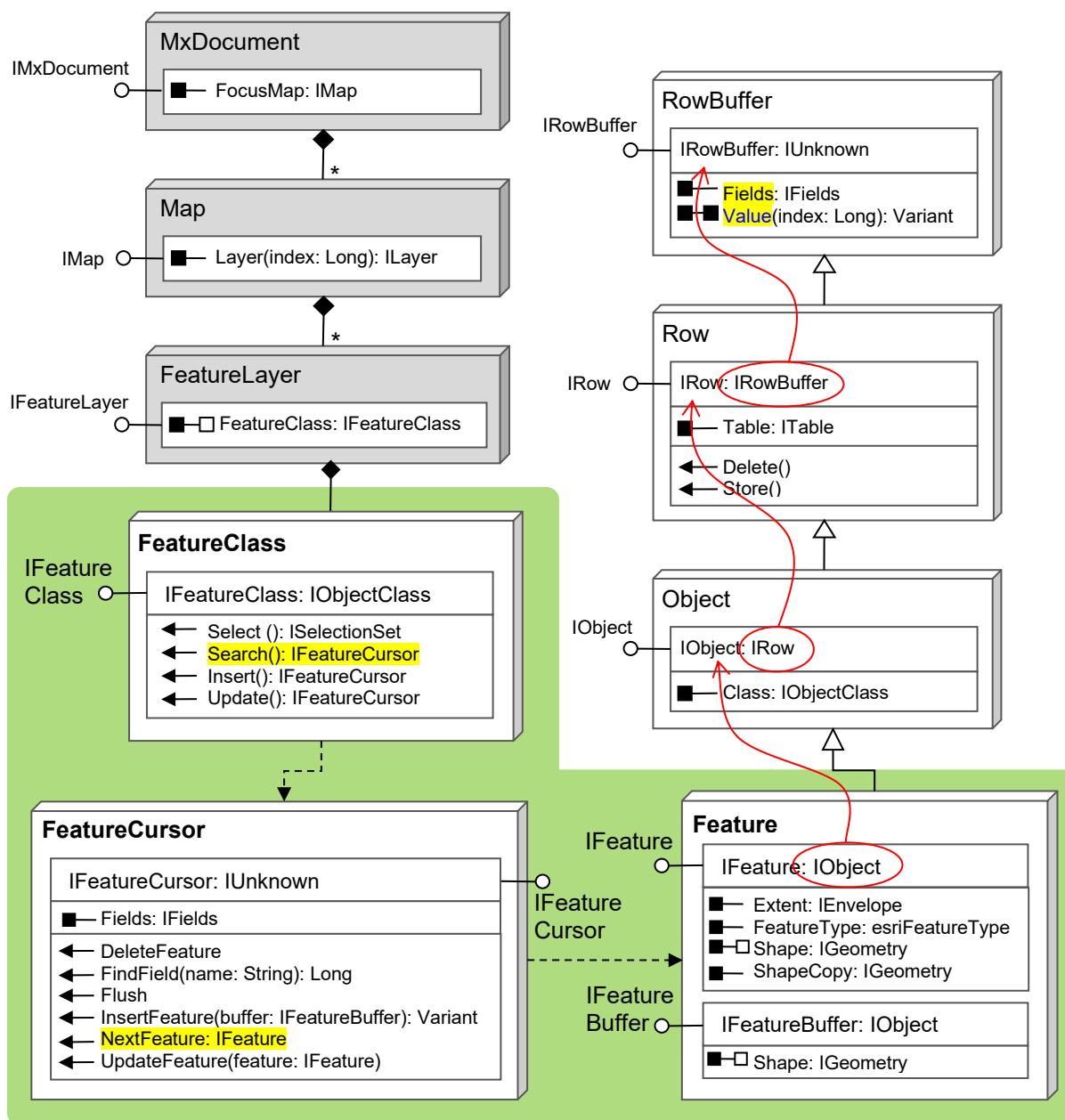


Figure 1 Object model diagram for navigating from map document to feature data

1. Accessing attribute values

At first glance, Figure 1 looks a little complicated. But you are already familiar with the left part of the diagram which shows how to hop step by step from the document object (with IMxDocument interface) to feature class object. You also played with the bottom part (green shaded area) of the diagram in Chapter 12 in which you obtained a feature cursor object from a feature class, then retrieved a feature (a parcel) from the feature cursor, and finally you obtained an envelope object from the Extent property of a parcel feature. To access attribute data of features, we will continue to focus on this part of the diagram.

The IFeatureClass interface of the FeatureClass class provides three key methods: Search(), Update() and Insert(), each producing a feature cursor object (with IFeatureCursor interface) with an intended purpose explained in section 4.2 of Chapter 12.

To demonstrate the process of accessing attribute data of features, this section will guide you through the process of creating a custom identify tool that displays the attribute data of a feature you click on. A strategic solution to this problem is as the following:

- 1) use the Search() method to select a feature into a feature cursor;
- 2) get the feature object from the feature cursor, and finally;
- 3) retrieve attribute data from the feature object.

To implement the custom identify tool, a few new ArcObjects components need to be introduced.

1.1 SpatialFilter

The Search(queryFilter: IQueryFilter, recycle: Boolean) method requires two arguments: a query filter object and a Boolean value to indicate whether to rehydrate a single feature on each fetch while using the NextFeature() method. Chapter 12 suggests us to set True to the recycle parameter when you only need to read feature data, and to use False if you need to edit features. If the resulting cursor contains only one feature, it does not matter what value to pass to the recycle parameter. Since we only need to read the attribute data in the case of feature identifying, let's use True. For the query filter parameter, you have learned how to instantiate a query filter object from the QueryFilter co-class and then set a query expression to its WhereClause property. But to select a feature from a layer using mouse clicking, no query expression can be set. In fact, this is a spatial query operation in which a special type of query filter called spatial filter must be used. A spatial filter object can be instantiated from the SpatialFilter co-class (Figure 2).

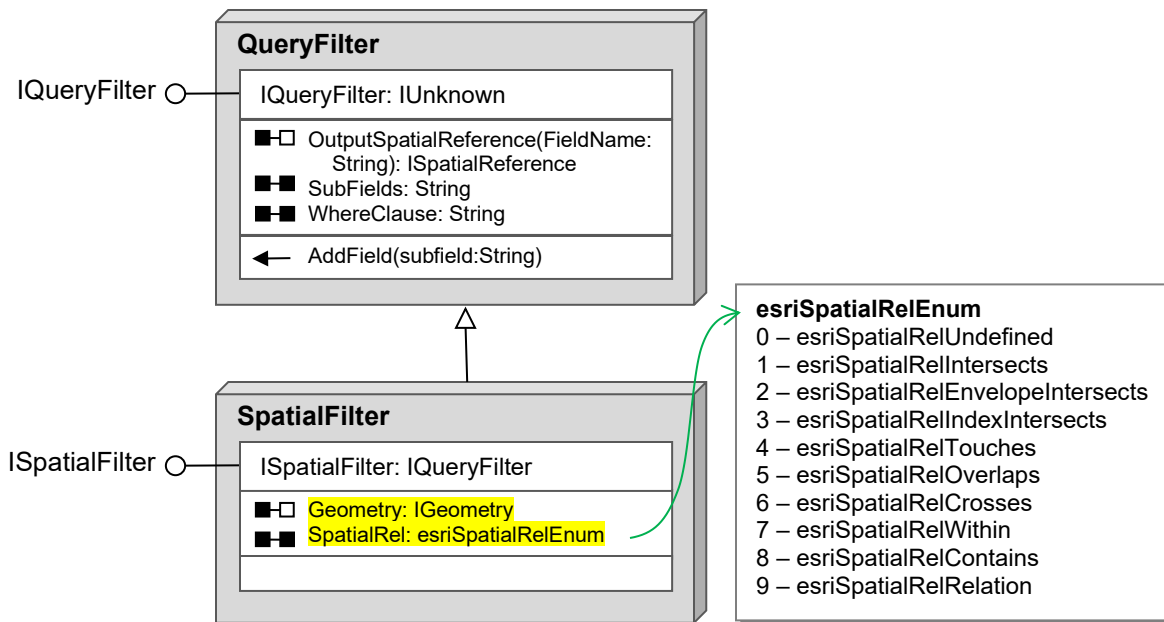


Figure 2 The QueryFilter and SpatialFilter co-classes
(Details in GeodatabaseObjectModel.pdf)

The SpatialFilter co-class inherits the QueryFilter co-class so that it inherits all the interfaces from its parent. Moreover, the ISpatialFilter interface inherits the IQueryFilter interface, therefore the ISpatialFilter interface takes on all properties and methods of the IQueryFilter including the WhereClause property. After instantiating a spatial filter object, we must set two key properties, Geometry and SpatialRel, in order to make the object functional. The Geometry property allows us to set a geometry object (selector) that is used to select features from a target feature layer. The SpatialRel property specifies a spatial relationship between the selector and target features. For example, you can set a point object to the Geometry property as a selector. Depending on the geometry type (point, line or polygon) of the selector and the target feature layer, a proper spatial relationship must be specified. The ESRI has defined an enumeration of spatial relationships constants – the esriSpatialRelEnum (Figure 2). All the constant values in the enumeration are self-explanatory. To select a parcel (polygon feature) by a point selector, for example, you can set constant esriSpatialRelWithin to the SpatialRel property.

Suppose you have a map document object represented by variable *mxDoc* (pointing to IMxDocument interface) and a polygon feature class, say parcel, represented by variable *fclass* (pointing to IFeatureClass interface), you can create a spatial filter object that uses a mouse clicking point to select a polygon feature using the following code:



```

' get a point object from the mouse location
Dim p As IPoint = mxDoc.CurrentLocation

' create a new spatial filter object from the SpatialFilter co-class
Dim sFilter As ISpatialFilter = New SpatialFilter
sFilter.Geometry = p
sFilter.SpatialRel = esriSpatialRelEnum.esriSpatialRelWithin

' create a feature cursor by calling the Search() method of the feature class
  
```

```
Dim fCursor As IFeatureCursor = fclass.Search(sFilter, True)

' call the NextFeature() method to get a feature from the cursor
Dim feature As IFeature = fCursor.NextFeature()
```

1.2 Fields and Value

Once you obtain a feature object, you can move on to retrieve its spatial and attribute data. Please look at the right-hand side part of Figure 1. The Feature regular class (at bottom) inherits a series of classes from Object via Row to RowBuffer (on top). Therefore, a feature object inherits all interfaces of every super classes. Moreover, with a closer look at the diagram, you will find that the IFeature interface inherits the IObject interface which inherits the IRow interface which inherits the IRowBuffer interface. This sequence of interface inheritance means that the IFeature interface finally takes on all the properties and methods of every super interfaces. As a result, the IFeature interface also has properties Fields and Value which are originally on the IRowBuffer interface.

The Value property takes an index argument as input and returns the value of an attribute of the specified index. For example, if you have a polygon feature object represented by variable *feature* and its Area attribute is the fifth field, then you can use *feature.Value(4)* to get the area value of this feature.

The Fields property of a feature returns a Fields objects with IFields interface (Figure 3). A Fields object is a container that holds all field objects. It has a FieldCount property that indicates the number of fields it contains, i.e. the number of fields in the attribute table. The Field property takes an index argument as input and returns a field object. If you do not know the index of a specific field, you can call the FindField() method by passing a field name to it to find the index. If a field of the specified name exists in the fields container, the FindField() method returns a non-negative integer. Otherwise, it returns -1.

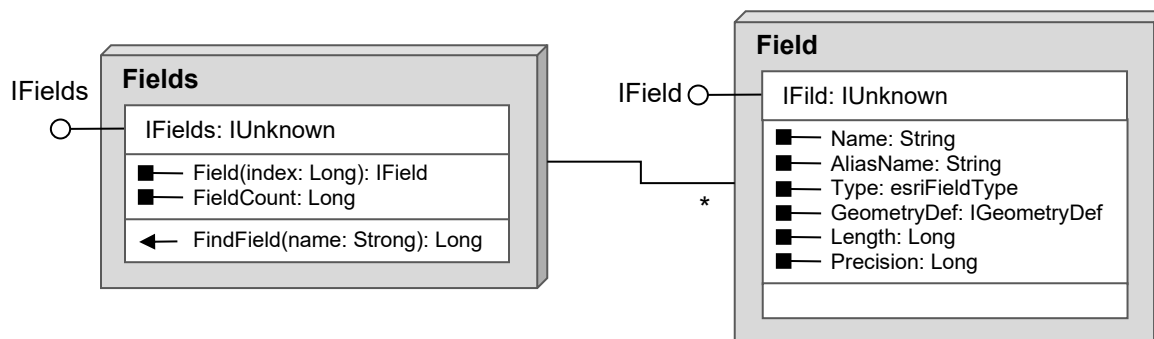


Figure 3 The Fields and Field co-classes
(Details on page 1 of GeodatabaseObjectModel.pdf)

A field object contains the metadata that describes a field (column or attribute) in an attribute table. A Field object has a number of properties. Figure 3 shows a few frequently used properties on the IField interface of the Field class. To get the name of a field, you can call the Name property. To find the data type (such as Long, Double, and String etc.) of a field, you can call the Type property. The left-ended barbell icons in the diagram indicate that all the properties on the IField interface are read-only properties.

Given a feature object represented by variable *feature* pointing to IFeature interface, you can get the fields object from it, and then the name and value of any specific field by the following code:



```
Dim fields As IFields = feature.Fields ' gets the fields container
Dim field As IField = fields.Field(2) ' gets the 3rd field from fields
Dim fieldName As String = field.Name ' gets the name of the 3rd field

' display the name and value of the 3rd field
MsgBox(fieldName & ": " & feature.Value(2).ToString)
```

Suppose you have a list box control named *lstInfo*, you can use a repetition to display the name and value of each field of the feature in the list box with the following code:



```
Dim fields As IFields = feature.Fields ' gets the fields container
Dim fieldName As String ' declares a variable for field names
For i As Integer = 0 To fields.FieldCount - 1
    fieldName = fields.Field(i).Name
    lstInfo.Items.Add(fieldName & ": " & feature.Value(i).ToString)
Next
```

1.3 Creating your own Identify tool

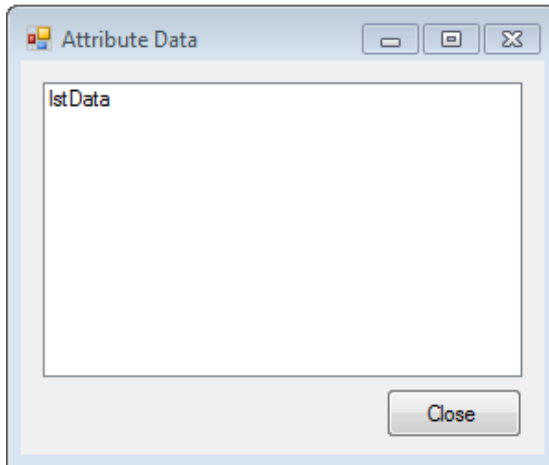
Example 1: So far you have learned all techniques needed to create a custom identify tool to display attribute data of polygon features. Perform the following tasks to create your own identify tool in the CityGIS project.

- Task 1: Create a new add-in tool in your CityGIS project and add it into the toolbar. You will use this tool to select features from a feature layer.
- Task 2: Create a Windows form in the project and add a list box control on the form. This form will be used to display attribute data of a selected feature.
- Task 3: Create a method (a public procedure) in the windows form class to display attribute data of a feature passed in as an argument.
- Task 4: Write code in the OnMouseDown event procedure to select a feature and then pass the selected feature object to the windows form to display attribute data.

Please follow steps below to accomplish these tasks.



- 1) Create an add-in tool with the following settings:
 - a. Name: "IdentifyTool"
 - b. Caption and tooltip text: "Identify Features"
 - c. Image:  in folder Images on the CD-ROM
- 2) Add the tool into your CityGIS toolbar by editing the Config.esriaddin XML document
- 3) Create a Windows form with the following settings and GUI (Figure 4):



Control	Property Settings
Form	Name: frmIdentify Caption: Attribute Data
ListBox	Name: lstData Anchor: Top, Bottom, Left, Right
Button	Name: btnClose Caption: Close Anchor: Bottom, Right

Figure 4 The attribute data windows form user interface

- 4) Enter the following code in the click event procedure of the Close button on the Windows form:

```
Me.Close()
```

- 5) Import the Geodatabase namespace in the header of the Windows form module:

```
Imports ESRI.ArcGIS.Geodatabase
```

- 6) Add the following method (a public sub procedure) into the frmIdentify class:

```
Public Sub ShowAttribute(feature As IFeature)
    Dim fields As IFields = feature.Fields ' gets the fields container object
    Dim fieldName As String ' declares a variable for field names
    Dim strValue As String ' declares a variable for field values
    For i As Integer = 0 To fields.FieldCount - 1
        fieldName = fields.Field(i).Name
        If fieldName.ToUpper() = "SHAPE" Then Continue For ' skips the Shape field
        strValue = feature.Value(i).ToString()
        lstData.Items.Add(fieldName & ": " & strValue)
    Next
End Sub
```

This procedure takes a feature object (with IFeature interface) as an input argument, and uses a repetition to add the name and value of each field into a list box. Most of the code is explained in section 1.2. Please pay attention to the If block inside the repetition. This line is used to prevent the SHAPE field from being displayed because the SHAPE field contains a geometric object which is neither a string nor a number. If the name of a field is "SHAPE", keywords *Continue For* direct the program flow to the Next statement of the repetition. ToUpper() is a method of a string object that converts all letter to upper case. It is used in the If condition to make the string comparison case insensitive. ToString() is a method of many data types (including numbers) that converts the data into a string.

- 7) Open the add-in IdentifyTool class module. Add the following line in the constructor (Public Sub New()) block to set a hand icon to the mouse cursor of the tool:

```
Me.Cursor = Windows.Forms.Cursors.Hand
```

Word “cursor” here refers to the icon of the mouse pointer. It is not the ArcObjects container object that holds features or rows.

- 8) Add the following imports statements into the header of the of the add-in IdentifyTool code module:

```
Imports ESRI.ArcGIS.ArcMapUI
Imports ESRI.ArcGIS.Carto
Imports ESRI.ArcGIS.Geodatabase
Imports ESRI.ArcGIS.Geometry
```

- 9) Add the following OnMouseDown() event procedure into the class. To do so, type the following code in the IdentifyTool class:

```
Protected Overrides Sub OnMouseDown(
```

As you keep typing, the Visual Studio IntelliSense continues to provide more and more pertinent suggestions. When the OnMouseDown event is focused in the IntelliSense, you can hit the Enter key to accept the suggestion to complete. The event procedure should look like the following:

```
Protected Overrides Sub OnMouseDown(arg As
                                ESRI.ArcGIS.Desktop.AddIns.Tool.MouseEventArgs)

    MyBase.OnMouseDown(arg)
End Sub
```

Then you can enter the following code in the event procedure.

```
' get the map document
Dim mxDoc As IMxDocument = My.ArcMap.Document

' get the selected layer in the TOC
' (when you test the program, make sure a polygon layer is selected)
Dim layer As ILayer = mxDoc.SelectedLayer

' QI to the IFeatureLayer interface. Function TryCast() is used to prevent crash.
' if an image layer is highlighted in the TOC, TryCast() returns object Nothing
Dim flayer As IFeatureLayer2 = TryCast(layer, IFeatureLayer2)
If flayer Is Nothing Then Return

' make sure the selected layer is a polygon layer
If flayer.ShapeType <>
    ESRI.ArcGIS.Geometry.esriGeometryType.esriGeometryPolygon Then Return

' get a feature class object from the feature layer object
Dim fclass As IFeatureClass = flayer.FeatureClass

' get a point object from mouse location
Dim p As IPoint = mxDoc.CurrentLocation

' instantiate a new spatial filter object from the SpatialFilter co-class
```

```

Dim sFilter As ISpatialFilter = New SpatialFilter
sFilter.Geometry = p
sFilter.SpatialRel = esriSpatialRelEnum.esriSpatialRelWithin

' create a feature cursor by calling the Search() method of the feature class
Dim fCursor As IFeatureCursor = fclass.Search(sFilter, False)

' call the NextFeature method to get a feature from the cursor
Dim feature As IFeature = fCursor.NextFeature()
If feature Is Nothing Then Return

' instantiate a Windows form object from the frmIdentify class
Dim frmInfo As frmIdentify = New frmIdentify
frmInfo.ShowAttribute(feature) ' calls the ShowAttributes() method
frmInfo.ShowDialog()          ' calls the ShowDialog method to display it

```

The second line of code calls the SelectedLayer property on the IMxDocument interface to get a layer object with ILayer interface. Since the selected layer may not be a feature layer, the third line uses the TryCast() function to cast it into a feature layer object with IFeatureLayer interface. If the selected layer is really a feature layer, the casting will be successful and a feature layer object will be assigned to variable *fLayer*. If the selected layer is an image layer, the TryCast() will return a Nothing object because a raster layer cannot be casted into a feature layer. The fourth line uses an If block to check if a feature layer object is derived. If not, it forces the program flow to exit the procedure.

Starting from line 6 (`Dim fclass As ...`), the program gets the feature class object from the feature layer, gets a point object from the mouse location, and creates a spatial filter object using the point object as a selector. Then it calls the Search() method of the feature class by passing the spatial filter to it to create a feature cursor. Finally, it calls the NextFeature() method to retrieve a feature object from the cursor. To display the attribute data of the feature, the program instantiates a new Windows form from the frmIdentify class and calls its ShowAttribute() method by passing the feature object to the method (see step 6).

To select polygon features using a point feature in this example, only one feature can be selected into the feature cursor at a time. In some spatial selection operations, for example, when a selector is a rectangle object and spatial relationship is “intersects”, multiple features may be selected into the cursor.

- 10) Run the program to test your custom Identify tool. Make sure you select (highlight) a polygon feature layer in the map before identifying features.

Challenge questions:

- a) This identify tool can be used to identify polygon features so far. Think about how you may make it to identify line features and point features.
- b) When a polygon feature is identified, only its attribute values are displayed but the feature is not highlighted. To improve your feature identification tool, think about how to highlight an identified feature.

2. Editing Attribute Values

Example 2: The parcel feature class in the Flagstaff geodatabase has a field named ParcelArea. But many values of this field are not correct. You will create an add-in button in your CityGIS toolbar to compute the area of each parcel in acres for the Parcel feature class. Before creating the tool, please copy the personal geodatabase file “Flagstaff.mdb” from folder Data on the CD-ROM to your local drive and take a look at the attribute table of the Parcel layer using ArcMap.

2.1 Problem analysis

This should not be a very difficult task since you have learned all ArcObjects components needed to accomplish this task. Please take one more look at the simplified object model diagram in Figure 1 to analyze the problem.

First, from the map document (IMxDocument) you can easily hop to the feature class object of the parcel layer.



```
Dim mxDoc As IMxDocument = My.ArcMap.Document ' gets the document
Dim map As IMap = mxDoc.Maps.Item(0) ' gets the Land map
Dim flayer As IFeatureLayer = map.Layer(1) ' gets the Parcel layer (2nd layer)
Dim fclass As IFeatureClass = flayer.FeatureClass ' gets the layer's feature class
```

Second, a feature class object has three methods, Search(), Update() and Insert(), all producing a feature cursor object. Since you need to update the ParcelArea field value for every feature in the feature class, you must use the Update() method to obtain a feature cursor this time. The Update() method needs a query filter object as input. But in this problem, since you need a feature cursor that contains all features of the layer, you can simply pass object Nothing to the method. For the second parameter (recycling), you must use False because you are going to edit feature values.

```
' get a feature cursor that contains all features of the feature class
Dim fcursor As IFeatureCursor = fclass.Update(Nothing, False)
```

Third, call the NextFeature() method in a repetition to obtain each feature from the feature cursor.

```
' get a feature from the feature cursor
Dim feature As IFeature = fcursor.NextFeature
```

Fourth, get a geometry object (with IGeometry interface) from the ShapeCopy property of each feature. You may also to get a feature's geometry from the Shape property. But be aware that the former is a read-only property and the latter is a two-way property. If you want to edit the shape or to set a new geometry to a feature, you must use the latter. When you get a shape object from the ShapeCopy or Shape property, you get an IGeometry interface. You have to use the TryCast() function to cast it into a polygon object (with IPolygon interface) in order to use attributes or methods of the polygon class. Since the Area property of a polygon object is on the IArea interface, you need to use QI to switch to the IArea interface in order to obtain the polygon's area.



```

' declare a few variables for use
Dim shape As IGeometry
Dim poly As IPolygon
Dim polyArea As IArea
Dim area As Double

```

The following four lines will be executed repeatedly for each feature in a repetition:

```

shape = feature.ShapeCopy      ' gets the shape (IGeometry) of a feature
poly = TryCast(shape, IPolygon) ' casts the shape from IGeometry to a IPolygon
polyArea = poly                ' QI to the IArea interface
area = polyArea.Area / 43560    ' gets the polygon area (1 acre = 43560 sq. ft.)

```

Fifth, the IFeature interface of a feature object inherits the Fields property and Value property from the IRowBuffer interface. You can use these properties to set area values to the ParcelArea field. Suppose you do not know the index of the ParcelArea field in the Fields container, you can call the FindField() method on the IFields interface of the Fields object to find the index in order to set values to the Value property.



```

' get the index of the ParcelArea field
Dim x As Integer = feature.Fields.FindField("ParcelArea")

' set value to the ParcelArea field
feature.Value(x) = area ' this line can be executed repeatedly in a loop

```

Finally, after setting a value to the ParcelArea field, you need to call the UpdateFeature() method of the feature cursor object to save the change to database.

```

fcursor.UpdateFeature(feature) ' updates the feature cursor

```



In the second step, you use the Update() method of the feature class object to create a feature cursor because you want to edit attribute values of all features. With a feature cursor obtained from the Update() method, method UpdateFeature() of the cursor object must be used to save the change made to each feature. If in the second step you call the Search() method to select a subset of features, these features can also be edited, but you must use the Store() method on the IFeature interface of the feature object interface (inherited from the IRow interface) to save the change.

2.2 Implementation



- 1) Copy the personal geodatabase Flagstaff.mdb from the CD-ROM to your hard drive, and update the file path in the AddLayersButton class.
- 2) Create an add-in button in the CityGIS project.
 Name: "CalculateParcelAreaButton"
 Caption and tooltip text: "Calculate Parcel Area"
 Image:  in CityGIS\Images on the CD-ROM
- 3) Add the button into the CityGIS toolbar by editing the Config.esriaddin XML document
- 4) Use the following import statements to import four namespaces in the header section of the CalculateParcelButton class module:

```
Imports ESRI.ArcGIS.ArcMapUI
Imports ESRI.ArcGIS.Carto
Imports ESRI.ArcGIS.Geodatabase
Imports ESRI.ArcGIS.Geometry
```

5) Implement the OnClick() event procedure by the following code:

```
Protected Overrides Sub OnClick()
    Dim mxDoc As IMxDocument = My.ArcMap.Document ' gets the document
    Dim map As IMap = mxDoc.Maps.Item(0) ' gets the Land map
    Dim flayer As IFeatureLayer = map.Layer(1) ' gets the Parcel layer

    ' get the layer's feature class
    Dim fclass As IFeatureClass = flayer.FeatureClass
    ' get the feature cursor from the feature class
    Dim fcursor As IFeatureCursor = fclass.Update(Nothing, False)

    ' get the first feature
    Dim feature As IFeature = fcursor.NextFeature

    ' get the index of the ParcelArea field
    Dim x As Integer = feature.Fields.FindField("ParcelArea")

    ' declare a few variables for use in a loop below
    Dim shape As IGeometry
    Dim poly As IPolygon
    Dim polyArea As IArea
    Dim area As Double

    ' iterate through all features
    Do While feature IsNot Nothing
        shape = feature.ShapeCopy ' gets the shape of a feature
        poly = TryCast(shape, IPolygon) ' converts the shape to a polygon
        polyArea = poly ' QI to get the IArea interface
        area = polyArea.Area / 43560 ' gets the area of the polygon
        feature.Value(x) = area ' sets value to the ParcelArea field
        fcursor.UpdateFeature(feature) ' updates the feature cursor
        feature = fcursor.NextFeature ' gets the next feature
    Loop
    MsgBox("Field 'ParcelArea' has been updated successfully!")
End Sub
```

Run the program to test the tool. Check values of the ParcelArea field after the computation. If you want to test the tool again, you can use the Field Calculator of ArcMap to change values in the field to 0.

To successfully test the tool, the parcel layer must be the second layer in the top map. Otherwise, the program will crash. If you have completed Exercise 3 of Chapter 12, you can call the GetLayer() function to obtain the Parcel layer no matter where it is located in the document.

3. Adding new features into feature classes

Example 3: In Section 3 of Chapter 11, you created a tool that displays bus routes from data in a GPS file. The tool could be more useful if it can convert the GPS data into features in a geodatabase. Adding this capability would involve a few more ArcObjects programming techniques including creating a blank feature class in a geodatabase, creating new features from data in the text file, and inserting features into the feature class. This section will show you these techniques and lead you to achieve the goal.

Since the emphasis of this section is to create new features and adding them into feature classes, code for database management tasks including creating new feature datasets and feature classes will be provided to you.

A general procedure to implement the capability is illustrated by Figure 5. The flowchart is composed of four major parts: 1) validating user input; 2) creating a blank feature class in a user specified feature dataset (shaded in pink); 3) creating new features and inserting them into the feature class (shaded in blue); and 4) displaying the new feature class in ArcMap.

The first and last parts are relatively simple. To validate the user input, you may simply check if the user has provided necessary information or not. Required user inputs include a GPS data file, an output personal geodatabase name, a feature dataset name and a feature class name. Displaying the resulting bus route feature class should also be simple to you since you have performed that many times. The following will focus on parts 2 and 3.

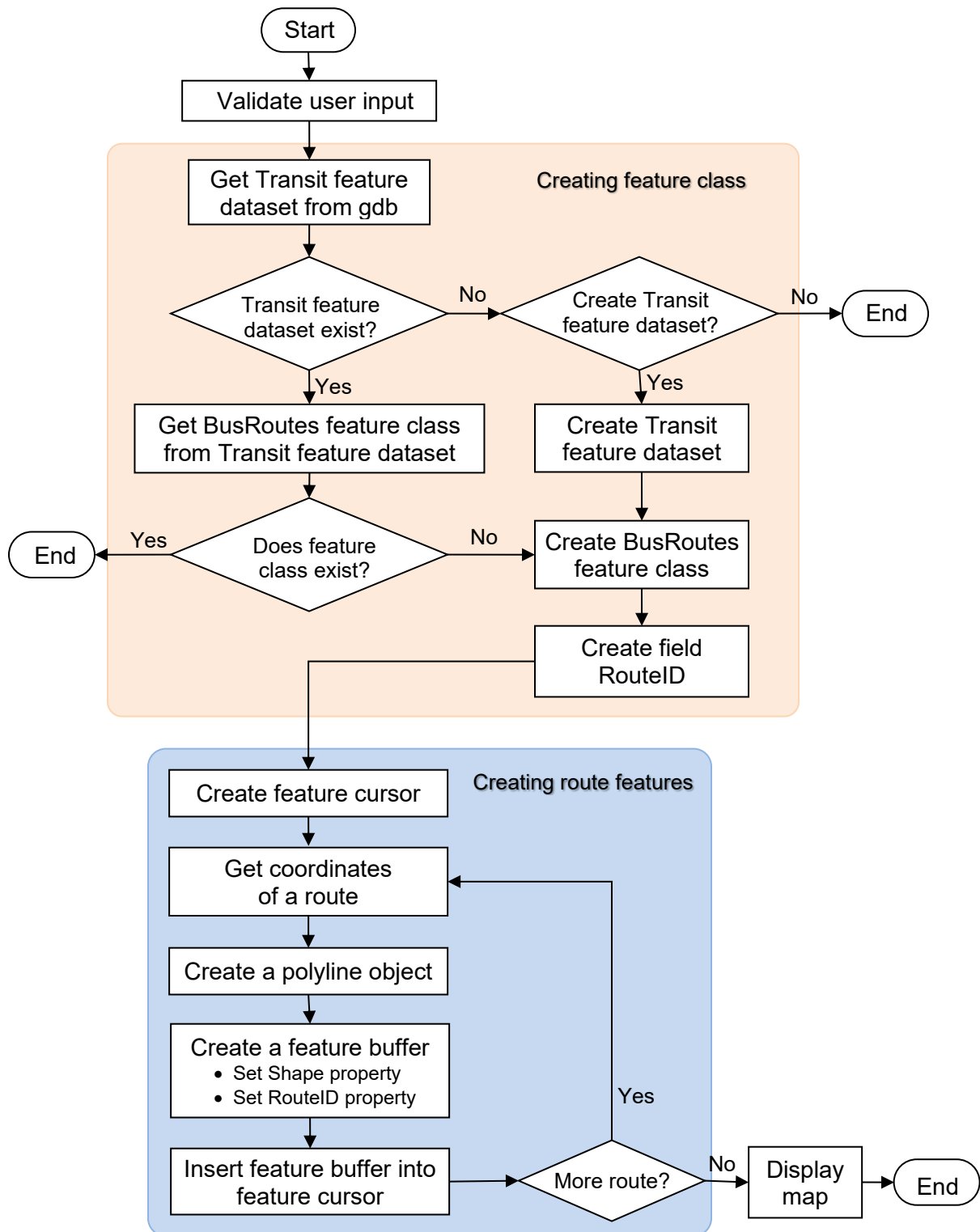


Figure 5 Flowchart for converting a GPS data file into a feature class (BusRoutes) in a feature dataset (Transit) in a personal geodatabase (Flagstaff.mdb)

3.1 Manipulating a geodatabase

Part 2 is to create a blank bus route feature class in a specific feature dataset of a geodatabase. Although this part involves several database management techniques that are not introduced, a geodatabase management class will be provided to you to accomplish the tasks. According to Figure 5, the program will try to get a Transit feature dataset from a personal geodatabase (Flagstaff.mdb). If the feature dataset exists, it will try to get a bus route feature class from the dataset. If the specified bus route feature class exists, it will send a message to notify the user and then finishes. Otherwise, it creates a blank bus route feature class in the feature dataset. In the previous step, if the Transit feature dataset does not exist in the geodatabase, it will ask the user whether to exit the program or to create a new Transit feature dataset. If the user chooses the later, then it creates a new Transit feature dataset and a new bus route feature class in the feature dataset. After creating a blank bus route feature class, the program will add a new integer field named "RouteID" in the new feature class. The resulting feature class will have few attributes including Shape, ObjectID, Shape_Length and the "RouteID" field.

Geodatabase management tasks to be performed in Part 2 include 1) getting a feature dataset object from a geodatabase; 2) getting a feature class object from a feature dataset; 3) creating a feature dataset in a geodatabase; 4) creating a new feature class in a feature dataset, as well as 5) creating a new field (RouteID) in the feature class. A geodatabase management class is provided to you to accomplish the first four tasks. Please find file "GDBManager.vb" in folder Source_Code on the CD-ROM. You can import the file into your CityGIS project and used the GDBManager class to instantiate database manager objects. Figure 6 shows the interface of the GDBManager class.

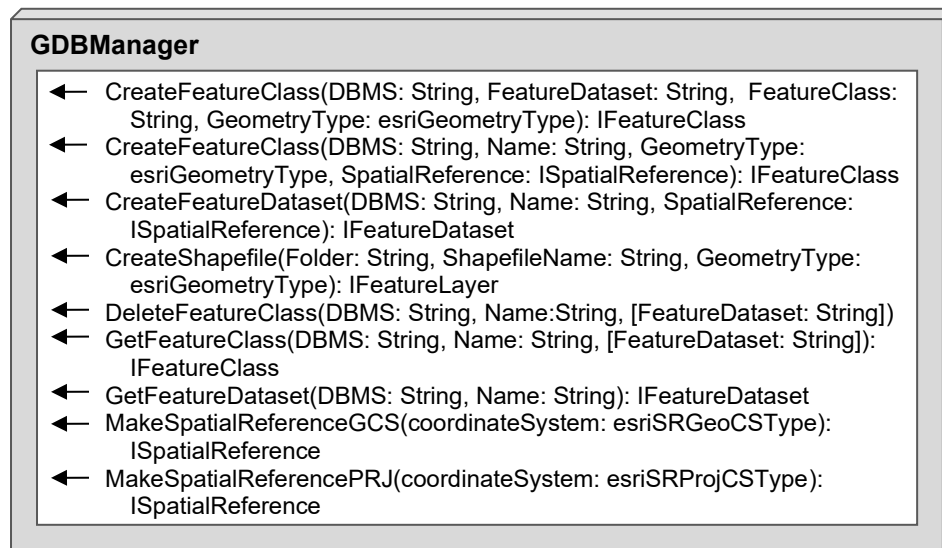


Figure 6 The GDBManager coclass

The GDBManager is a co-class having only one unnamed interface. This class has nine methods but you will use five of them to perform tasks in this example: GetFeatureDataset(), GetFeatureClass(), CreateFeatureDataset(), CreateFeatureClass() (with one overloaded), and MakeSpatialReferenceGCS(). The following demonstrates how to use these methods.



```

' create a new gdb manager object from user class GDBManager
Dim gdbmgr As New GDBManager
  
```

```

' to get a user specified feature dataset from a geodatabase
Dim fDataset As IFeatureDataset = gdbmgr.GetFeatureDataset(dbmsName, datasetName)

' to make a GCS spatial reference object
Dim spref As ISpatialReference
spref = gdbmgr.MakeSpatialReferenceGCS(esriSRGeoCSType.esriSRGeoCS_NAD1983)

' to create a new feature dataset in a geodatabase
Dim fDataset As IFeatureDataset
fDataset = gdbmgr.CreateFeatureDataset(dbmsName, datasetName, spref)

' to create a new feature class in the feature dataset (the first overloading method)
Dim fClass As IFeatureClass
fClass = gdbmgr.CreateFeatureClass(dbmsName, datasetName, featureClassName,
                                   esriGeometryType.esriGeometryPolyline)

```

Once a new bus route feature class is created, you will add a new field “RouteID” into its fields. Although a method could have been created in the geodatabase manager class for this task, to let you practice the skill of creating fields by writing code, such a method is not provided.

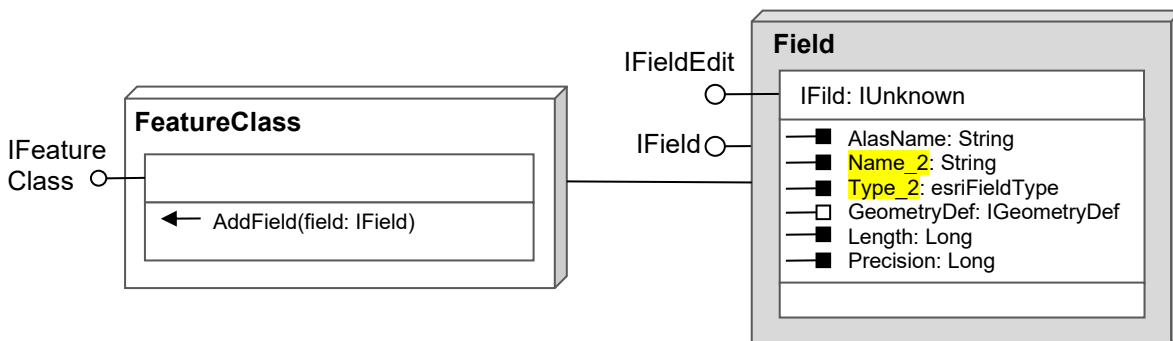


Figure 7 Object model diagram for creating a field object and adding it into a feature class
(Details on page 1 of GeoDatabaseObjectModel.pdf)

Creating a new field in a feature class is quite straightforward as indicated by the object model in Figure 7. First, you can use the Field co-class to instantiate a new field object. Then given a feature class object, you can call the AddField() method of the feature class to add the new field object into it. Once you have instantiated a new field object, you must set at least two properties on the IFieldEdit interface: 1) a field name to the Name_2 property and 2) a data type to the Type_2 property. If the field is a string type field, you may set an integer value to the Length property to define the maximum number of characters allowed for the field. If the field is floating point number field, you may set a Precision property to specify the number of significant digits. Finally, if the field is a spatial data field, you must set the GeometryDef property. Since field RouteID is an Integer field, you only need to set the Name_2 and Type_2 properties.

The IFieldEdit interface has the same set of properties as the IField interface (Figure 3). The difference between these two interfaces is that properties on the IField interface are read-only properties, but properties on the IFieldEdit interface are write-only properties. This difference is indicated by the different barbell icons.

Suppose you are provided with a feature class object, you can create a sub procedure to append a new RouteID field into it as the following. Please enter the following procedure in the BusRoutesDockableWindow class.



```
Private Sub CreateRouteIDField(ByRef fClass As IFeatureClass)
    Dim field As IField = New Field ' instantiates a new field object
    Dim fieldEdit As IFieldEdit = field ' QI to the IFieldEdit interface
    fieldEdit.Name_2 = "RouteID" ' sets the Name_2 property
    fieldEdit.Type_2 = esriFieldType.esriFieldTypeInteger ' sets data type
    fClass.AddField(field) ' adds it into the feature class
End Sub
```

To draw your attention, parameter fClass is explicitly declared with a ByRef keyword although the keyword is optional in this case. Through the ByRef parameter, the procedure will send an updated feature class object back to the client. The ByRef keyword here is optional because the parameter (fClass) is an object instead of a simple data type such as integer or string. In VB programming, object arguments are always passed by reference (ByRef).

3.2 Adding new features into a feature class

Once you have created a new bus route feature class, you can add bus route features into it. A procedure for this task is laid out in Figure 5 (area shaded blue). Figure 8 further shows techniques for carrying out the task. 1) call the Insert() method of the feature class to create a feature cursor; 2) call the CreateFeatureBuffer() method of the feature class to create a new feature object with IFeatureBuffer interface; 3) set a geometry object, i.e. a polyline, to the Shape property and set other attribute values to the feature buffer; 4) call the InsertFeature() method of the feature cursor object to add the feature buffer into the feature class. Refer to the object model in Figure 8 for the relationships between related components.

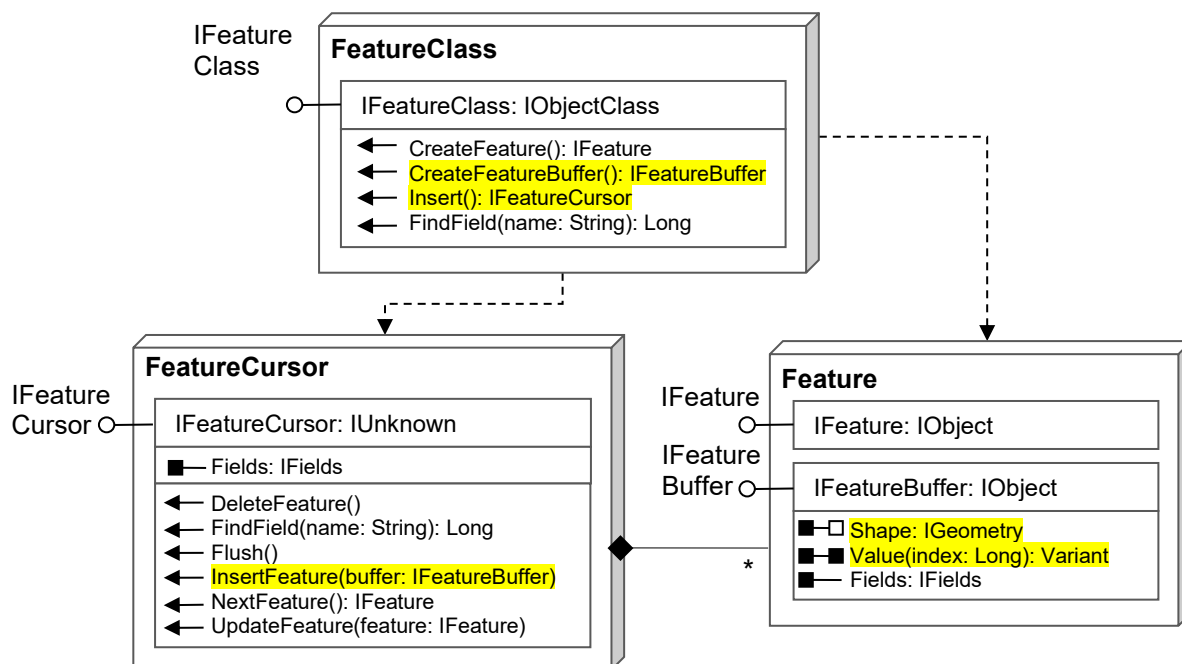


Figure 8 Object model diagram for adding features into a feature class

IFeatureBuffer is an interface of the Feature class. This interface allows you to set the Shape property and the Value property (for other attributes) of a feature object. Please recall that, in Section 3 of Chapter 11, you created a function that takes two coordinate arrays as input and returns a polyline objects. In this process, you will call that function to create a polyline for each bus route feature. When a new feature object is created, it resides in the computer memory. To save the feature into a feature class in a geodatabase, the InsertFeature() method of the feature cursor object will be called (Figure 8). Suppose you are provided with a feature class object represented by variable *fclass*, the process to creating bus route features and inserting them into the feature class can be illustrated by the following code.



```
' to create a feature cursor from a feature class
Dim fCursor As IFeatureCursor = fClass.Insert(False) ' False = not use buffering

' declare a couple of variables for use
Dim fBuffer As IFeatureBuffer = Nothing
Dim polyline As IPolyline = Nothing
```

Then you can perform the following in a loop that processes a bus route at each iteration.



```
' suppose you have two arrays, lon and lat, that contain coordinates of a route,
' you can call the CreatePolyline method to create a polyline object
polyline = CreatePolyline(lon, lat)

' create a new feature object with IFeatureBuffer interface
fBuffer = fClass.CreateFeatureBuffer ' the new feature is a blank object
fBuffer.Shape = polyline ' sets the Shape property to flesh out the feature
fBuffer.Value(2) = (a route id) ' sets RouteID value, suppose the field index is 2
fCursor.InsertFeature(fBuffer) ' inserts the new feature into feature class
```

3.3 Final assembly



Now, you have all skills needed for accomplishing all tasks. Please follow steps below to implement the bus route feature class creation functionality.

- 1) Find the GDBManager.vb class file in folder Source_Code on the CD-ROM and copy it into your CityGIS project folder (Figure 9).

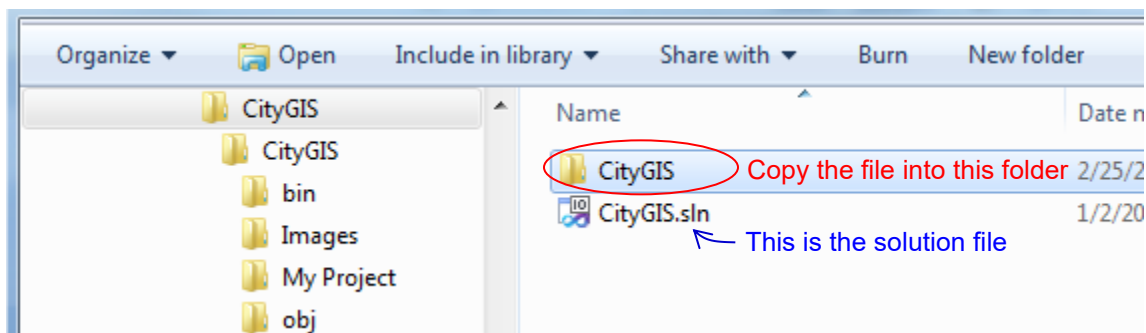


Figure 9 Save file GDBManager.vb in the project folder

- 2) Import the GDBManager.vb class into your CityGIS project: Right click on CityGIS in the Solution Explorer >> Add >> Existing Item >> GDBManager.vb. You will find the file added into the Solution Explorer after importing.
- 3) Modify the dockable window BusRoutesDockableWindow as the following (Figure 10):

- a. Create a group box (Name: GroupBox1; Text: Draw Bus Routes), and then move the combo box (cboRoutes) and two existing buttons (btnDrawRoute and btnClearMap) inside the group box.
- b. Create another group box (Name: GroupBox2; Text: Create Feature Class) and add the following new Windows controls in it:

Control	Name	Text
TextBox	txtDBMS	
TextBox	txtDataset	Transit
TextBox	txtFClass	BusRoutes
Button	btnBrowseDB	<<
Button	btnCreate	Create

Figure 10 Updated user interface of the bus route creation windows form

- 4) Add the following import statements in the BusRoutesDockableWindow class module if the namespaces are not imported.

```
Imports ESRI.ArcGIS.ArcMapUI
Imports ESRI.ArcGIS.Geodatabase
```

- 5) Implement the Click event of create button (btnCreate) as the following. Each part in the flowchart is carried out by a procedure.

```
Private Sub btnCreate_Click(...) Handles btnCreate.Click
    ' Part 1: validating user input by calling the ValidateInput() function
    Dim valid As Boolean = ValidateInput()
    If Not valid Then Return

    ' Part 2: creating a new BusRoutes feature class in the geodatabase
    Dim fClass As IFeatureClass = CreateNewFeatureClass()
    If fClass Is Nothing Then Return

    ' Part 3: Creating bus route (polyline) features in the feature class
    CreateBusRouteFeatures(fClass)

    ' Part 4: Displaying the layer in ArcMap
    DisplayFeatureClass(fClass)

    MsgBox("Layer '" & txtFClass.Text & "' was successfully created!")
End Sub

Private Function ValidateInput() As Boolean
```

```

If txtFile.Text = "" Then
    MsgBox.Show("Please select a bus route data file",
        "CityGIS",
        MsgBoxButtons.OK,
        MsgBoxIcon.Exclamation)
    Return False
End If

If txtDBMS.Text = "" Then
    MsgBox.Show("Please select a personal geodatabase for output",
        "CityGIS",
        MsgBoxButtons.OK,
        MsgBoxIcon.Exclamation)
    Return False
End If

If txtDataset.Text = "" Then
    MsgBox.Show("Please specify an output feature dataset",
        "CityGIS",
        MsgBoxButtons.OK,
        MsgBoxIcon.Exclamation)
    Return False
End If

If txtFClass.Text = "" Then
    MsgBox.Show("Please specify an output feature class name",
        "CityGIS",
        MsgBoxButtons.OK,
        MsgBoxIcon.Exclamation)
    Return False
End If

' at this point, all user input are valid
Return True
End Function

Private Function CreateNewFeatureClass() As IFeatureClass
    ' create a new gdb manager object from user class GDBManager
    Dim gdbmgr As New GDBManager

    ' get a the user specified feature dataset from the gdb
    Dim fDataset As IFeatureDataset = _
        gdbmgr.GetFeatureDataset(txtDBMS.Text, txtDataset.Text)

    ' declare a variable for feature class (IFeatureClass interface)
    Dim fClass As IFeatureClass = Nothing

    If fDataset Is Nothing Then ' feature dataset does not exist
        ' use a message box to ask the user whether to create a new feature dataset
        If Windows.Forms.MsgBox.Show(
            "The feature dataset does not exist." & Chr(13) & _
            "Do you want to create a new one?",
            "CityGIS",
            MsgBoxButtons.YesNo,
            MsgBoxIcon.Question,
            MsgBoxDefaultButton.Button1) = DialogResult.No Then
            Return Nothing ' if the user answers "No", then return Nothing
        Else ' if the user answers "Yes", create a series of new objects

```

```

        ' create a GCS spatial reference object for the new feature dataset
Dim spref As ISpatialReference = _
    gdbmgr.MakeSpatialReferenceGCS(esriSRGeoCSType.esriSRGeoCS_NAD1983)
    ' create a new feature dataset in the geodatabase
fDataset =
    gdbmgr.CreateFeatureDataset(txtDBMS.Text, txtDataset.Text, spref)
    ' create a blank feature class in the feature dataset
fClass = gdbmgr.CreateFeatureClass(txtDBMS.Text,
    txtDataset.Text,
    txtFClass.Text,
    esriGeometryType.esriGeometryPolyline)
    ' add a RouteID field in the new feature class
CreateRouteIDField(fClass)
End If
Else    ' if the specified feature dataset exists in the gdb
    ' try to get a feature class by name
fClass =
    gdbmgr.GetFeatureClass(txtDBMS.Text, txtFClass.Text, txtDataset.Text)
If fClass Is Nothing Then    ' if the specified feature class does not exist
    ' create a blank feature class in the feature dataset
fClass = gdbmgr.CreateFeatureClass(txtDBMS.Text,
    txtDataset.Text,
    txtFClass.Text,
    esriGeometryType.esriGeometryPolyline)
    ' add a RouteID field in the new feature class
CreateRouteIDField(fClass)
Else    ' otherwise, if the specified feature class already exist
    Windows.Forms.MessageBox.Show(
        "Feature class " & txtFClass.Text &
        " already exist." & Chr(13) & _
        "No new feature class was created.",
        "CityGIS",
        MessageBoxButtons.OK,
        MessageBoxIcon.Exclamation)
    Return Nothing    ' exit the sub and return Nothing
End If
End If
gdbmgr = Nothing

Return fClass    ' returns the new feature class
End Function

Private Sub CreateBusRouteFeatures(fClass As IFeatureClass)
    ' create a feature cursor from the feature class
Dim fCursor As IFeatureCursor = fClass.Insert(False)    ' do not use buffering
    ' get the index of the RouteID field in the attribute table
Dim xRouteID As Integer = fClass.FindField("RouteID")
    ' declare a couple of variables for use later
Dim fBuffer As IFeatureBuffer = Nothing
Dim polyline As IPolyline = Nothing

    ' create a new feature for each bus route
For i As Integer = 1 To 5    ' iterates for each route
    Dim lon() As Double = Nothing    ' declares an array for longitudes
    Dim lat() As Double = Nothing    ' declares an array for latitudes
    ' read coordinates of route i into arrays lat and lon
    ReadRouteCoordinates(i, lat, lon)
    ' create a polyline object from coordinates

```

```

        polyline = CreatePolyline(lon, lat)
        ' create a new feature object (with IFeatureBuffer interface)
        fBuffer = fClass.CreateFeatureBuffer
        fBuffer.Shape = polyline          ' sets the polyline to Shape property
        fBuffer.Value(xRouteID) = i      ' sets the RouteID field value
        fCursor.InsertFeature(fBuffer)    ' inserts the feature into feature class
    Next
    fCursor.Flush()
End Sub

Private Sub DisplayFeatureClass(fClass As IFeatureClass)
    Dim fLayer As IFeatureLayer = New FeatureLayer
    fLayer.Name = txtFClass.Text
    fLayer.FeatureClass = fClass

    Dim mxdoc As IMxDocument = My.ArcMap.Document
    Dim map As IMap = mxdoc.Maps.Item(1) ' get the 2nd (Transportation) map
    map.AddLayer(fLayer)

    Dim av As IActiveView = map          ' QI to IActiveView interface
    mxdoc.ActiveView = av                ' activate the Transportation map

    ' get the extent of the new feature layer from property AreaOfInterest
    Dim rect As IEnvelope = fLayer.AreaOfInterest
    rect.Expand(1.1, 1.1, True)          ' expand the envelope by 10%
    av.Extent = rect                    ' zoom the current map to the envelope

    av.Refresh()
    mxdoc.UpdateContents()
End Sub

```

- 6) Modify your ReadRouteCoordinates() procedure in the BusRoutesDockableWindow class by adding an integer parameter RouteNumber. Inside the Do repetition block, please change the If condition is to data(1) = CStr(RouteNumber).

```

Private Sub ReadRouteCoordinates(ByVal RouteNumber As Integer,
                                ByRef latitudes As Double(),
                                ByRef longitudes As Double())
    ...
    Do While Not sr.EndOfStream
        ...
        If data(1) = CStr(RouteNumber) Then ' this line is changed
            ...
        End If
    Loop
    ...
End Sub

```

After modifying this procedure, please change a previous call to the function in the Click event procedure of button btnDrawBusRoute as the following.

```
ReadRouteCoordinates(CInt(cboRoutes.Text), lats, lons)
```

- 7) Implement the Click event of button btnBrowseDB. Use this button to display an open file dialog for the user to select a personal geodatabase (*.mdb). Once a file is selected, the full path of the file is displayed in the geodatabase box (txtDBMS).

8) Run the program and test the bus route feature class creation functionality.

Summary and Highlights

By starting from the map document, you can travel all the way to individual features and their spatial and attribute data. Once you are able to access spatial and attribute data of individual features you can harness the power of GIS for spatial data management and spatial modeling.

The IFeatureClass interface of the FeatureClass class provides three key methods for creating cursors: Search() produces a cursor mainly for retrieving feature data; Update() produces a cursor intended for editing feature values; and Insert() produces a cursor for adding new features into a feature class.

To select features into a cursor, the Search() and Update() methods require a spatial filter object. The SpatialFilter is a child class of the QueryFilter class. After instantiate a spatial filter object, you must properly set its Geometry property and the SpatialRel property.

A feature object can be retrieved from a feature cursor by calling the NextFeature() method. The IFeature interface provides two properties to access the geometry of the feature: the read-only property ShapeCopy, and the read-and-write property Shape. The IFeature interface also provides a read-only Fields property that allows you to access metadata of all fields and a two-way Value property for you to access field values. The Fields property returns a fields object which is a container holding metadata of every field of the feature. The Value property requires an integer argument to indicate the index of a specific field.

You can instantiate new field objects from the Field class and set field properties on the IFieldEdit interface. After creating a field object, you can add it into a feature class by calling the AddField() method on the IFeatureClass interface.

To create a new feature for a feature class, you need to call the CreateFeatureBuffer() method on the IFeatureClass interface to create a feature with IFeatureBuffer interface. After instantiating a new feature object you can set a geometric object to the Shape property and set field values to the Value property on the IFeatureBuffer interface.

To add a new feature into a feature class, you can call the InsertFeature() method on the IFeatureCursor interface to permanently store the feature into the feature class.

Exercises

1. Answer the following questions.
 - 1) What method on IFeatureClass do you use to create feature cursors for reading attribute values?
 - 2) What method on IFeatureClass do you use to create feature cursors for editing features?
 - 3) What method on IFeatureClass do you use to create feature cursors for adding new features?
 - 4) The Search() and Update() methods on the IFeatureClass interface need a query filter object as input. If Nothing is sent to these query filter argument, no feature cursor will be returned by these methods. Is this statement true or false?
 - 5) What method do you used to obtain a feature from a feature cursor?
 - 6) What property do you use to read an attribute value from a feature?
 - 7) What are differences between a query filter and a spatial filter?
 - 8) What two properties you must set after instantiating a spatial filter?
 - 9) What does the SpatialRel property of a spatial filter object do?
 - 10) How do you find the number of attributes of a feature?
 - 11) How do you find the index of a specific field in the attribute table of a feature class?
 - 12) How do you find names of all fields in an attribute table of a feature class?
 - 13) In VB programming, what do keywords "Continue For" do?
 - 14) How do you get the selected layer from a map document?
 - 15) After setting a value to a field of a feature, what method of the feature cursor is used to make the change permanent?
 - 16) What interface do you use to set a name and data type after you create a new field object?
 - 17) What is the main difference between the IField interface and IFieldEdit interface of the Field class?
 - 18) In procedure calls, object arguments are always passed by reference. Is this statement true or false?
 - 19) When you create a new feature object, which interface do you use to set Shape property and attribute values?
 - 20) To save a new feature object permanently into a feature class, what method of the feature cursor do you call?
2. Create a new add-in tool named "BurnedAreaTool" in your CityGIS project and add it into the project toolbar. This tool will be used by the Fire Department to track forest fire burned areas. Once a burned area is tracked using this tool, an input box pops up to prompt for a date and time of the fire. After accepting a time, it saves the tracked polygon and the date/time information as a polygon feature in feature class "Fires" in personal geodatabase Flagstaff.mdb.

Before creating the add-in tool, please use ArcCatalog to create a feature dataset named "ForestFire" in the Flagstaff.mdb database with spatial reference of UTM Zone 12N on datum NAD 1983. Inside the ForestFire feature dataset, create a new polygon feature class named "Fires" with a field "Date_Time" of Date type.

Tips:

- Make sure you load the Fires feature class into the map before you test the tool. You may modify the AddLayersButton to load the feature class with other layers.

- You may call the `GetLayer()` function you created previously to get the Fires feature class when the fire layer is needed in this new add-in tool class.
- The `InputBox()` function supports default value (the third parameter). You may use `Now.ToString()` as a default date/time in the input box.
- VB has a built-in function `IsDate()`. You can use this function to validate the date/time entered by the user in order to avoid crashing when saving the value in a Date type field.